

So far

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Table 2.1 *Hierarchy of 2D coordinate transformations, listing the transformation name, its matrix form, the number of degrees of freedom, what geometric properties it preserves, and a mnemonic icon. Each transformation also preserves the properties listed in the rows below it, i.e., similarity preserves not only angles but also parallelism and straight lines. The 2×3 matrices are extended with a third $[\mathbf{0}^T \ 1]$ row to form a full 3×3 matrix for homogeneous coordinate transformations.*

Point operations

We may apply functions to modify pixel intensities independently of each other, such as the linear point process

$$g(\mathbf{x}) = aI(\mathbf{x}) + b$$

- a increases contrast (also called „gain“)
- b increases brightness (also called „bias“)
- We can add complexity by making a, b dependent on the image location:

$$g(\mathbf{x}) = a(\mathbf{x})I(\mathbf{x}) + b(\mathbf{x})$$

Then a and b in fact become gray level images themselves!

Linear filters

Given a kernel function $h : \mathbb{R}^D \rightarrow \mathbb{R}^C$, we can filter the image f into a new image g using the convolution operation

$$g(y, x) = \sum_{i,j} f(y - i, x - j) \cdot h(i, j)$$

In short form: $g = f * k$

Note that this refers to discrete images and kernels - in the continuous domain, the convolution uses integrals.

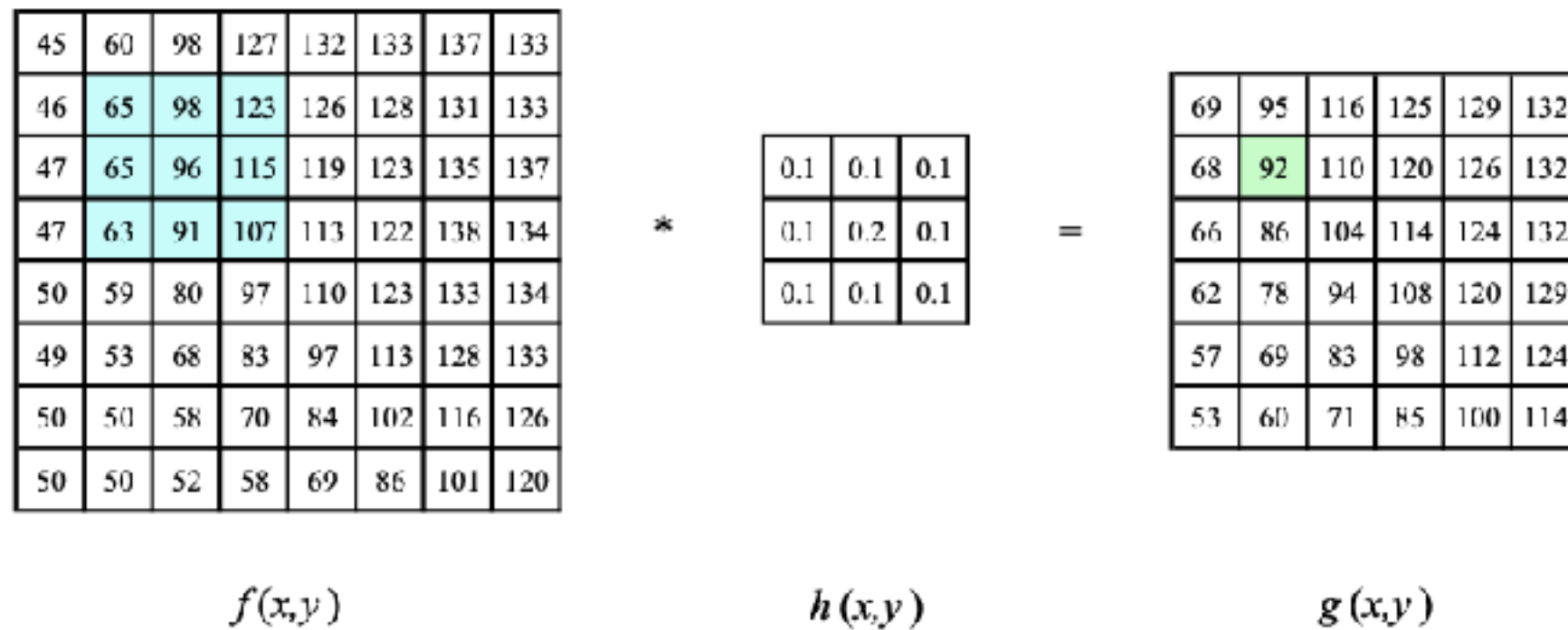


Figure 3.10 *Neighborhood filtering (convolution): The image on the left is convolved with the filter in the middle to yield the image on the right. The light blue pixels indicate the source neighborhood for the light green destination pixel.*

Key properties of linear filters

Linearity:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

Shift invariance: same behavior regardless of pixel location

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

Any linear, shift-invariant operator can be represented as a convolution

Box Filter

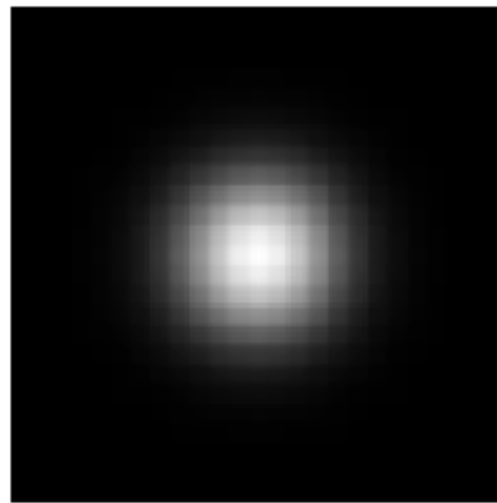
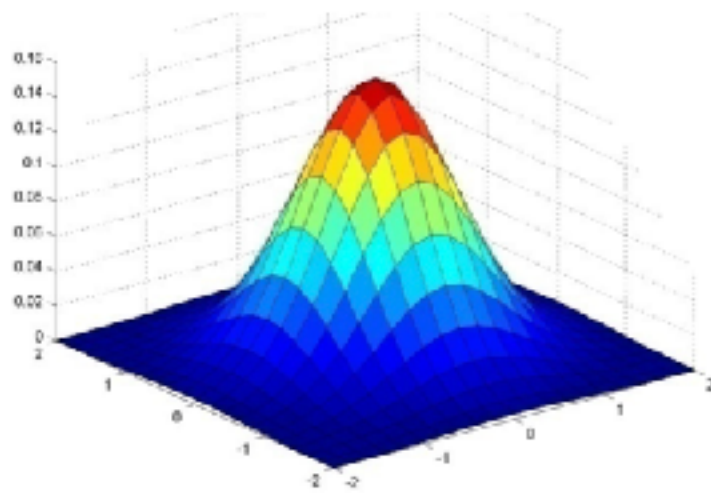
What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

$$\frac{1}{9} \begin{matrix} & g[\cdot, \cdot] \\ \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} \end{matrix}$$

Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Some more remarks on linear filters

Separability of the Gaussian filter

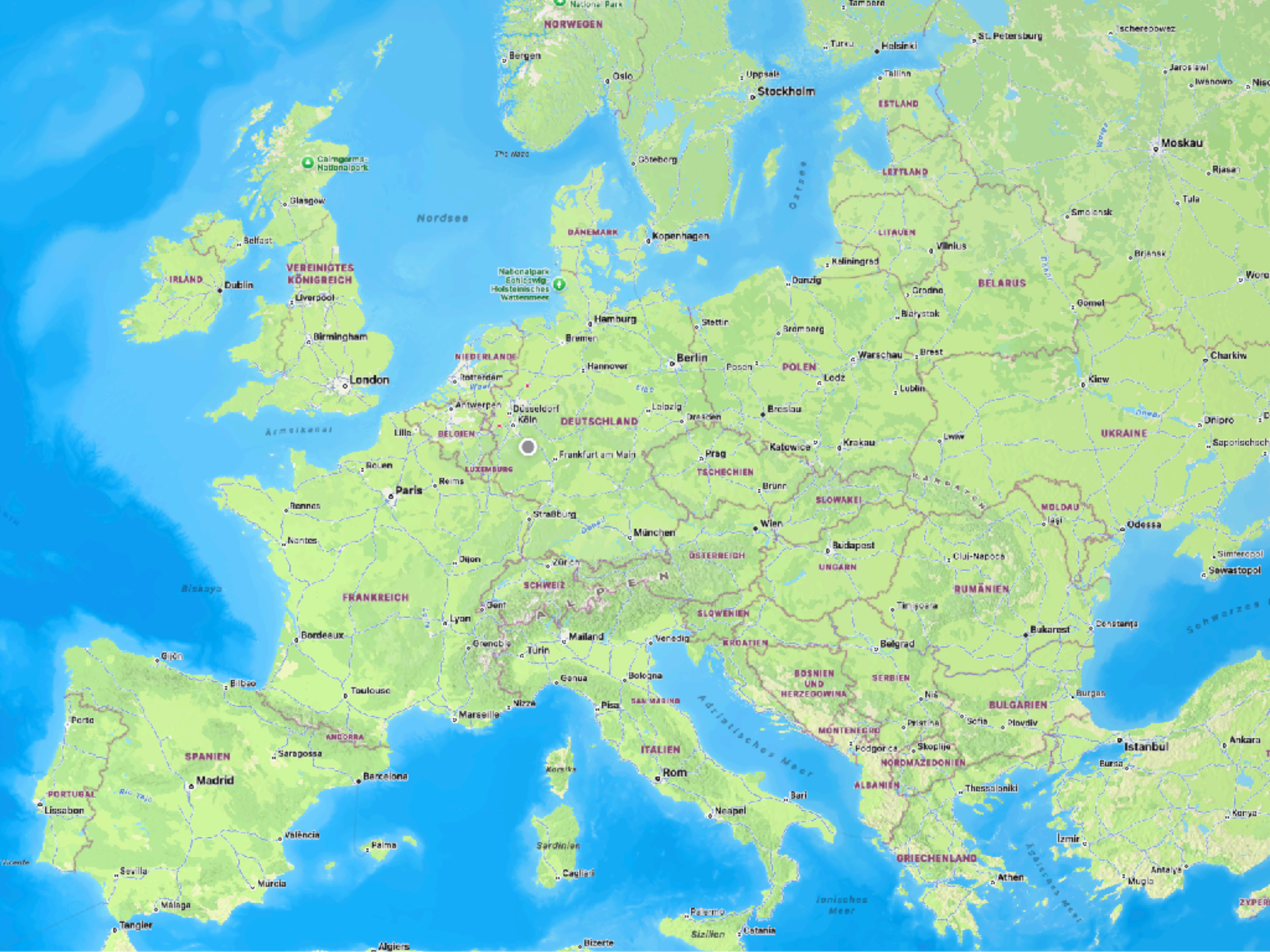
$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Median filters

- A **Median Filter** operates over a window by selecting the median intensity in the window.
- What advantage does a median filter have over a mean filter?
- Is a median filter a kind of convolution?



Features of interest

Int.J. Comput. Vis. (2011) 94:154–174
DOI 10.1007/s11263-010-0340-z

Coding Images with Local Features

Timo Dicksehl · Falko Schindler · Wolfgang Förstner

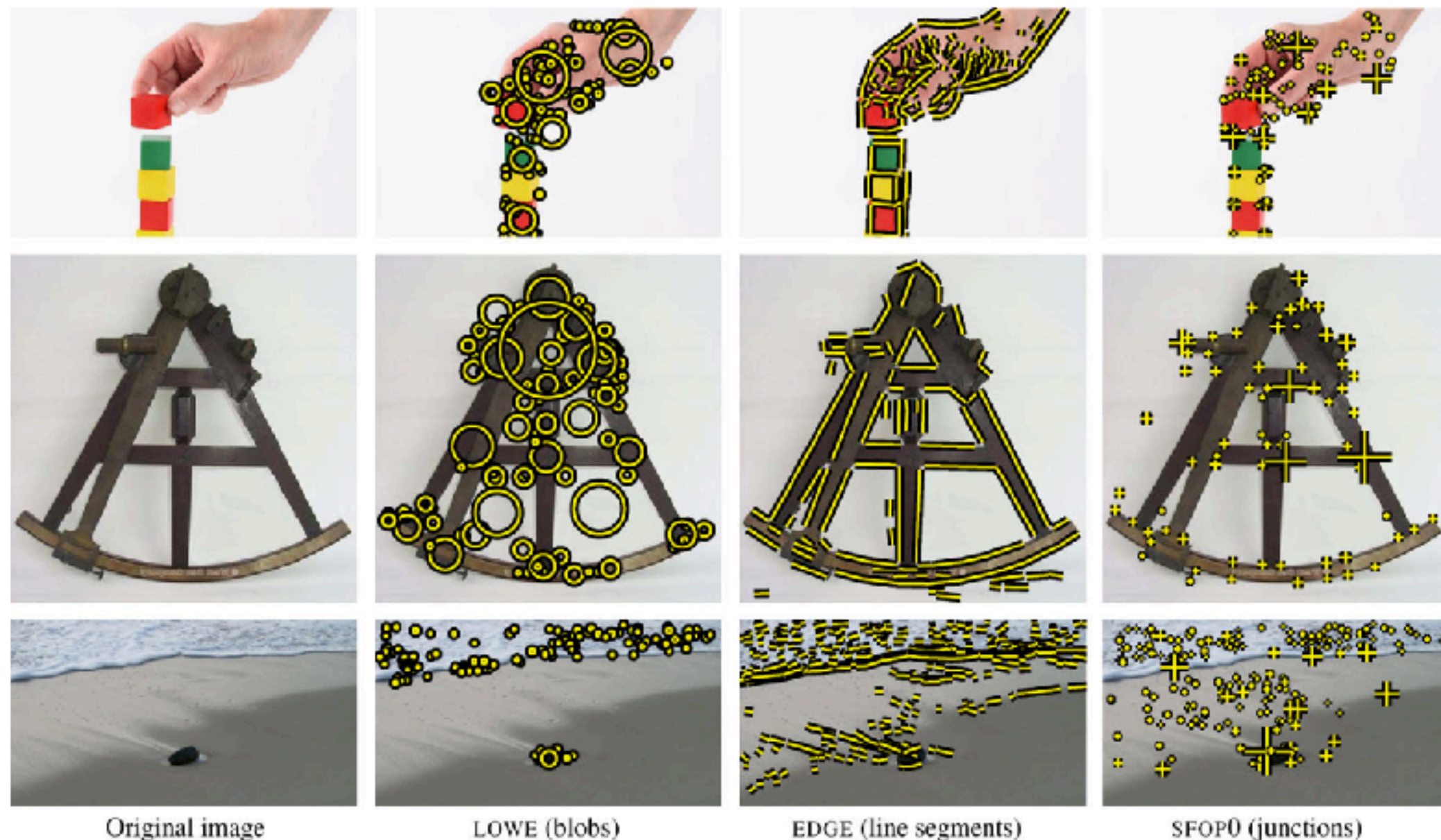
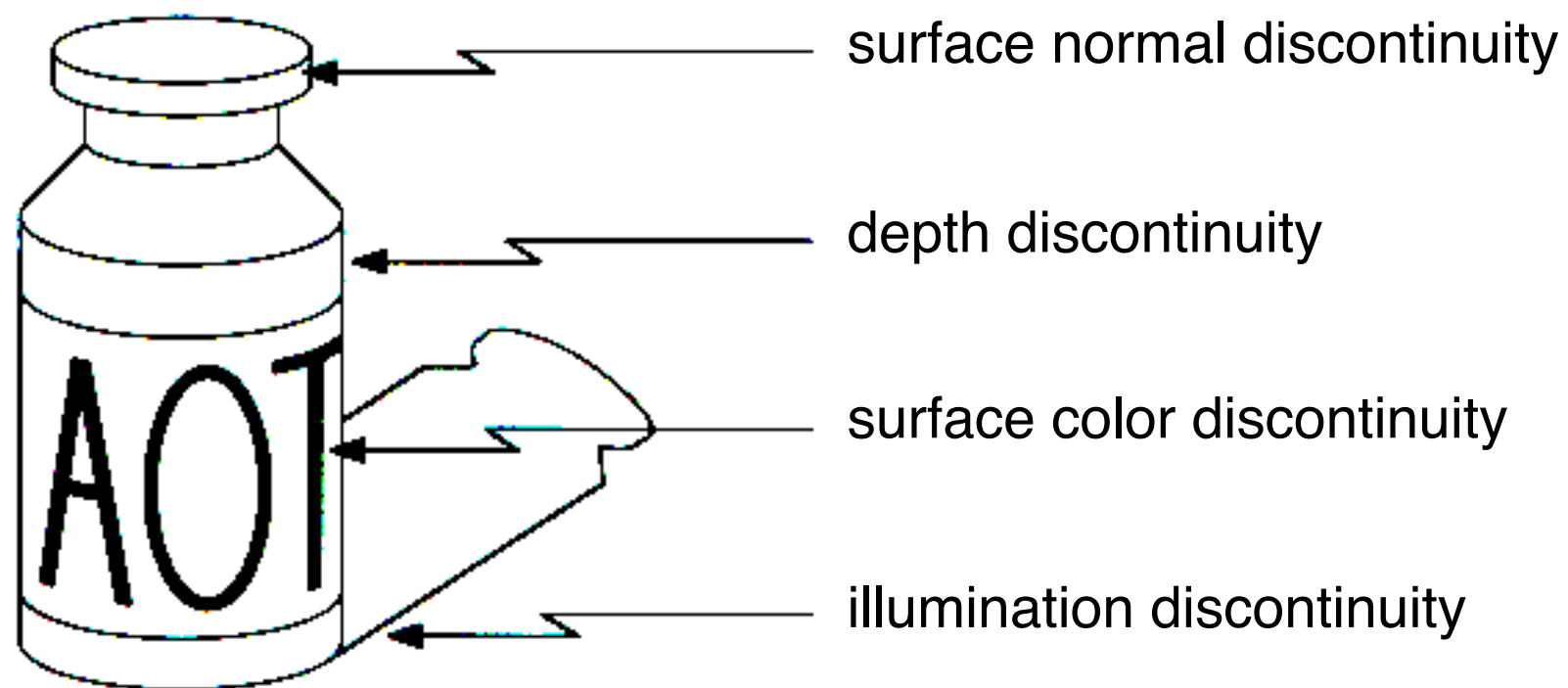


Fig. 1 Sets of extracted features on three example images. The feature detectors here are substantially different: LOWE (Lowe 2004) fires at dark and bright blobs, EDGE (Förstner 1994) yields straight edge

segments, and SFOP0 (Förstner et al. 2009, using $\alpha = 0$ in (7)) extracts junctions

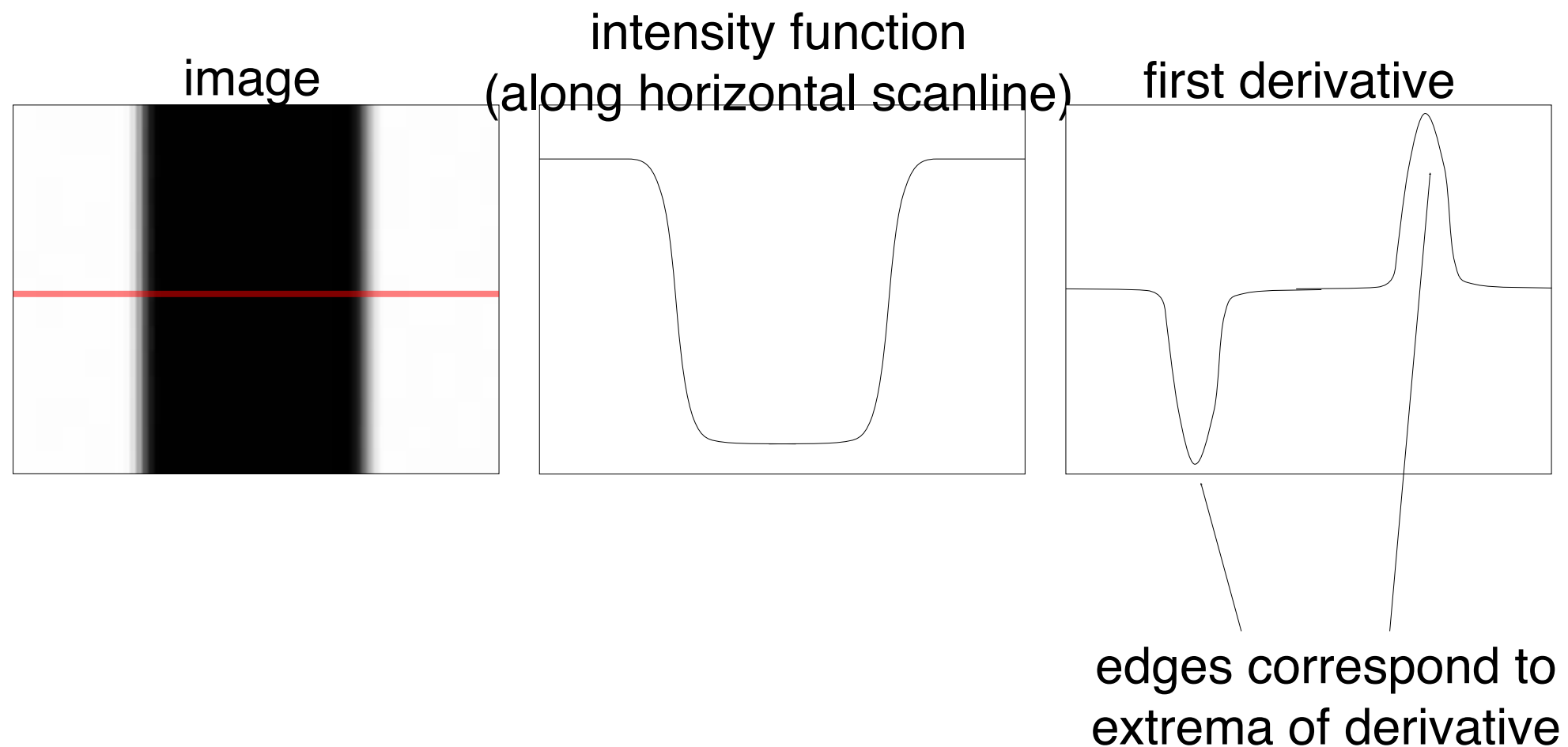
Edge-like features

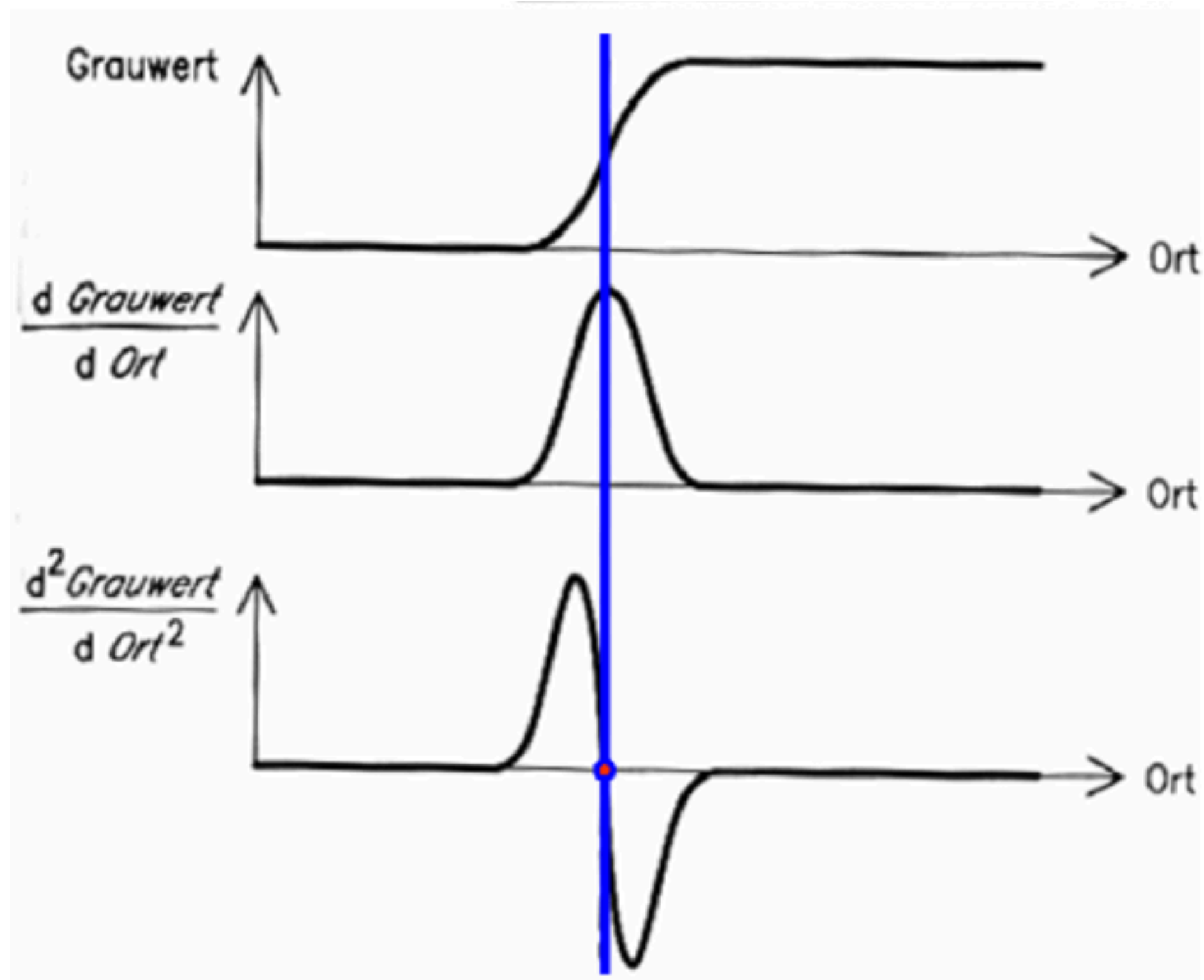
Origin of edges



Characterizing edges

- An edge is a place of *rapid change* in the image intensity function







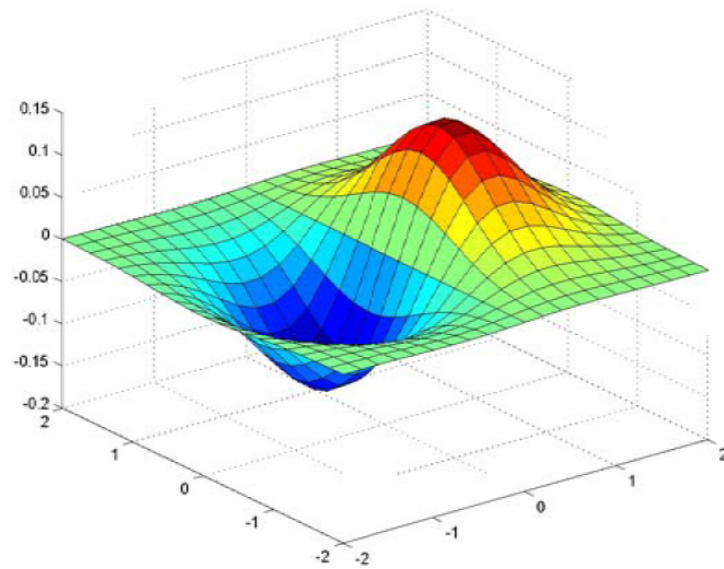
John F. Canny, UC Berkeley

A classical one: The Canny edge detector

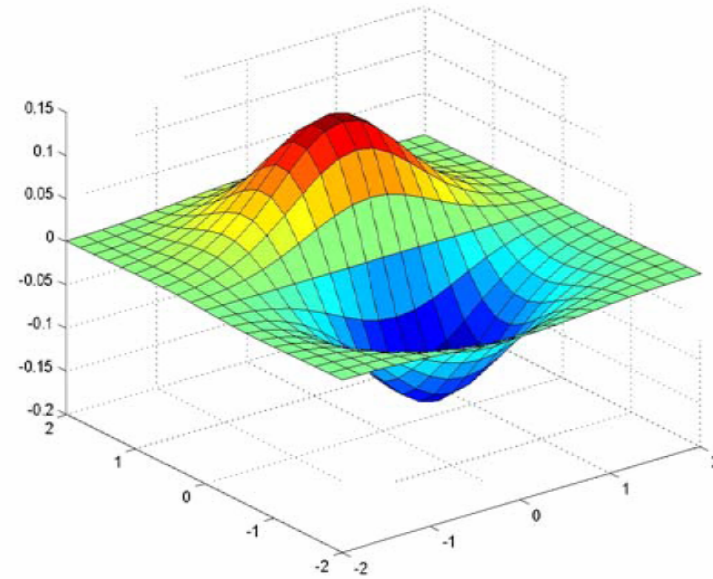
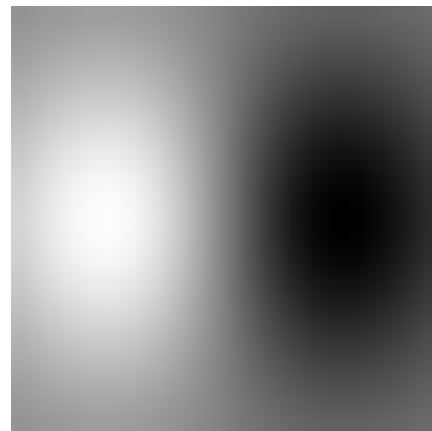
Canny edge detector

1. Filter image with x, y derivatives of Gaussian
 2. Find magnitude and orientation of gradient
 3. Non-maximum suppression:
 - Thin multi-pixel wide “ridges” down to single pixel width
 4. Thresholding and linking (hysteresis):
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them
- MATLAB: `edge(image, 'canny')`

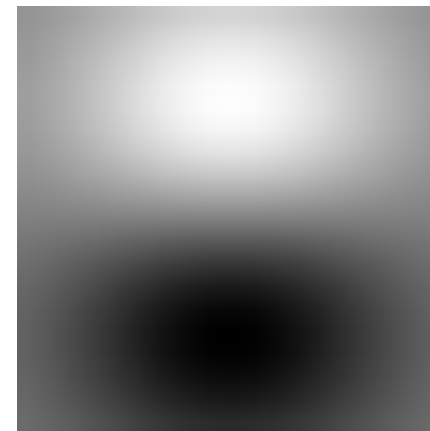
Derivative of Gaussian filter



x-direction



y-direction





X-Derivative of Gaussian



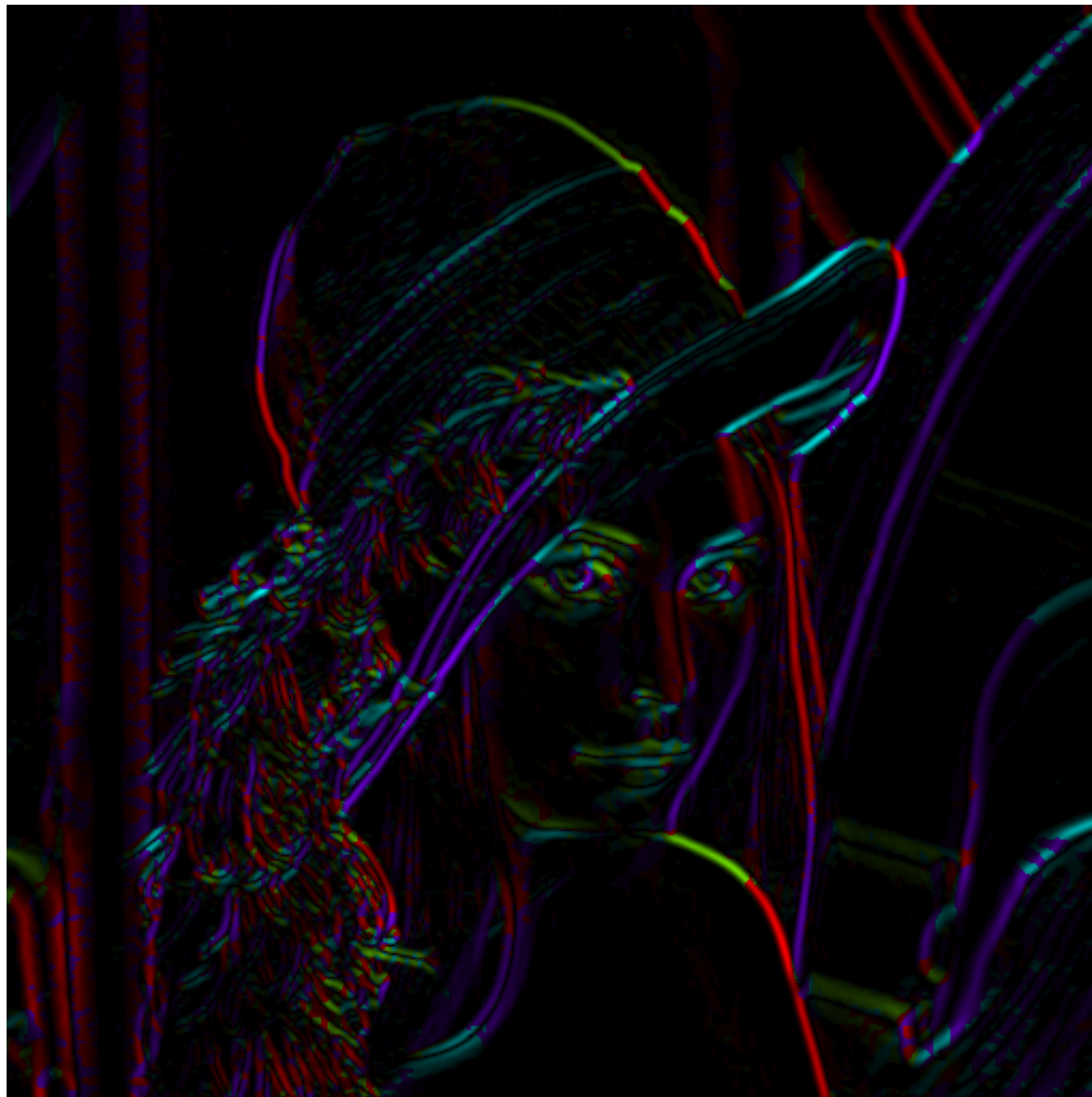
Y-Derivative of Gaussian



Gradient Magnitude

Get Orientation at Each Pixel

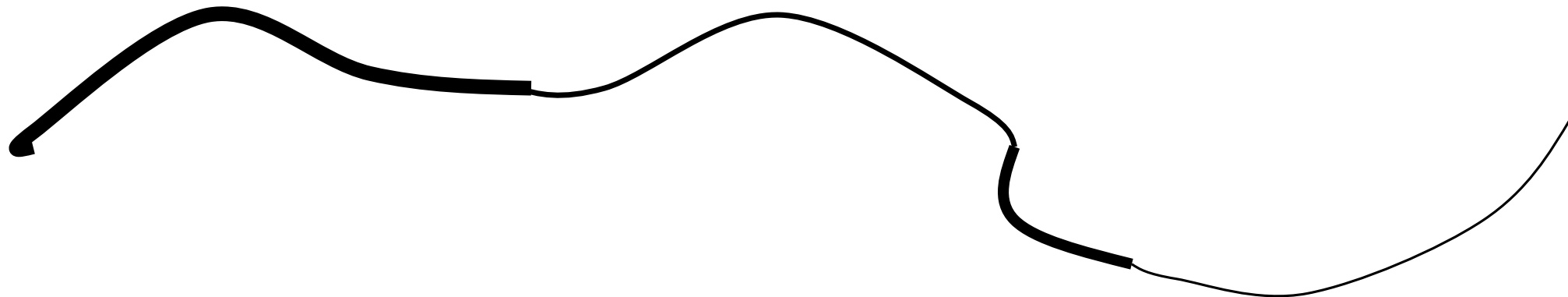
- Threshold at minimum level
- Get orientation



$$\text{theta} = \text{atan2}(\text{gy}, \text{gx})$$

Hysteresis thresholding

- Check that maximum value of gradient value is sufficiently large
 - drop-outs? use **hysteresis**
 - use a high threshold to start edge curves and a low threshold to continue them.



Final Canny Edges



The Laplacian filter

The Laplacian filter

- The Laplace operator is defined as the sum of all 2nd derivatives

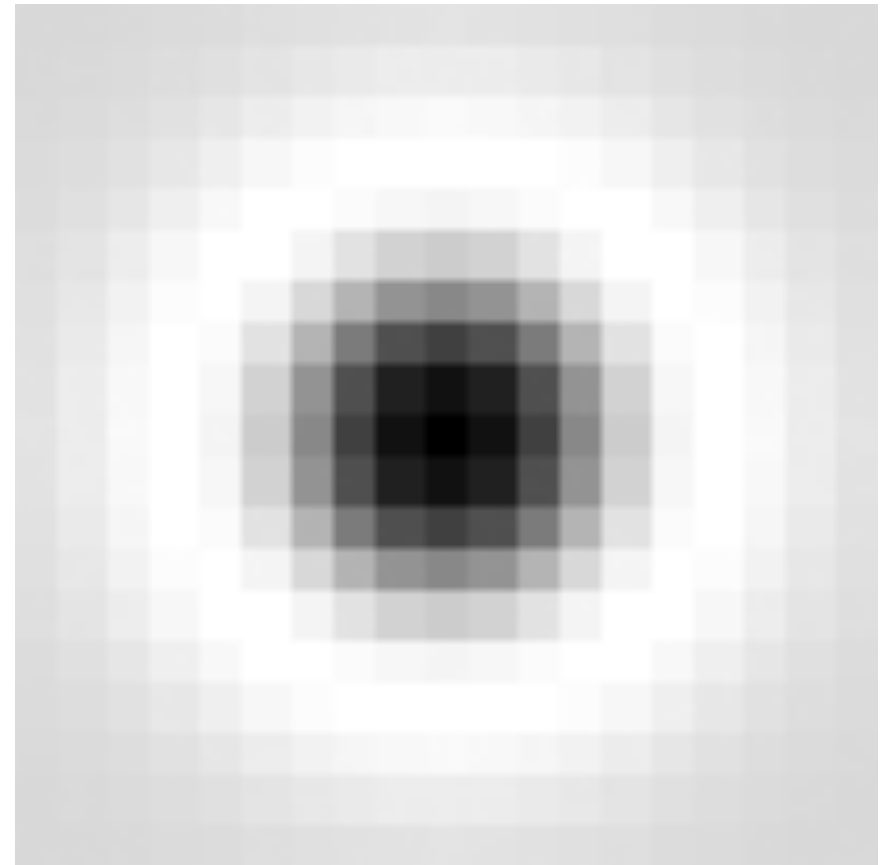
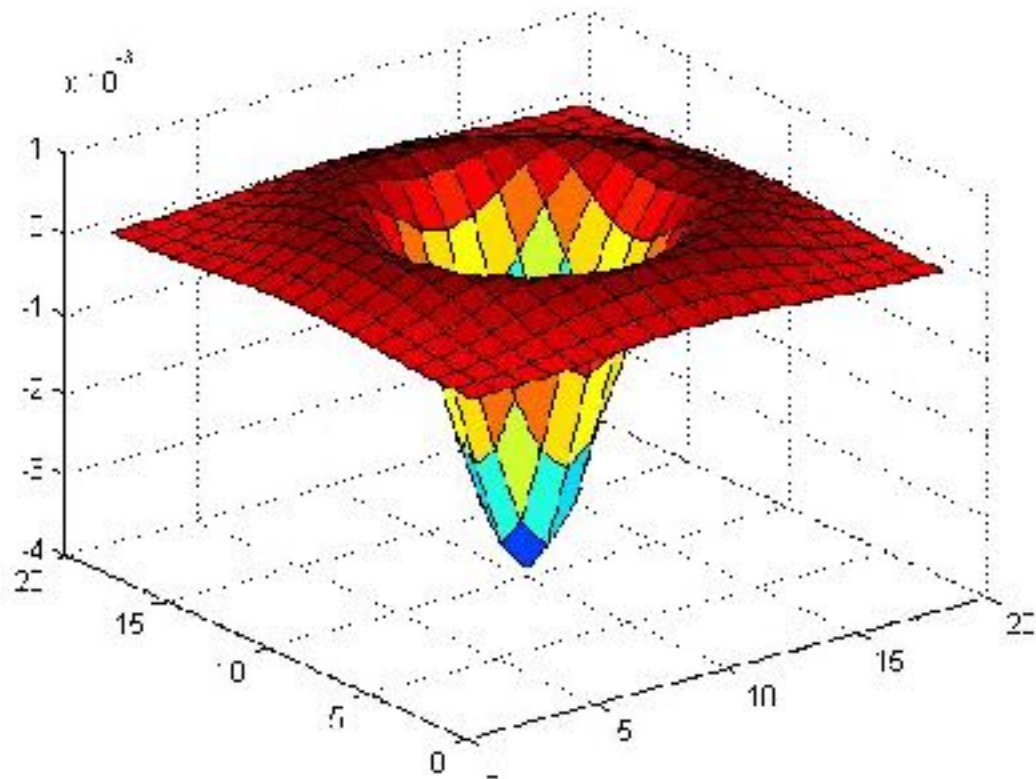
$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- The corresponding discrete convolution filter can be modeled in different ways:

$$\mathbf{D}_{xy}^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{D}_{xy}^2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Laplacian of Gaussian (LoG)



Laplace filtering

Kante

$$I_K = \begin{pmatrix} 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \\ \hline 120 & 120 & 120 & 120 & 120 \\ 120 & 120 & 120 & 120 & 120 \\ 120 & 120 & 120 & 120 & 120 \end{pmatrix}$$

Linie

$$I_L = \begin{pmatrix} 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \\ \hline 120 & 120 & 120 & 120 & 120 \\ \hline 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \end{pmatrix}$$

Ecke

$$I_E = \begin{pmatrix} 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \\ \hline 120 & 120 & 120 & 80 & 80 \\ 120 & 120 & 120 & 80 & 80 \\ 120 & 120 & 120 & 80 & 80 \end{pmatrix},$$

"Spot"

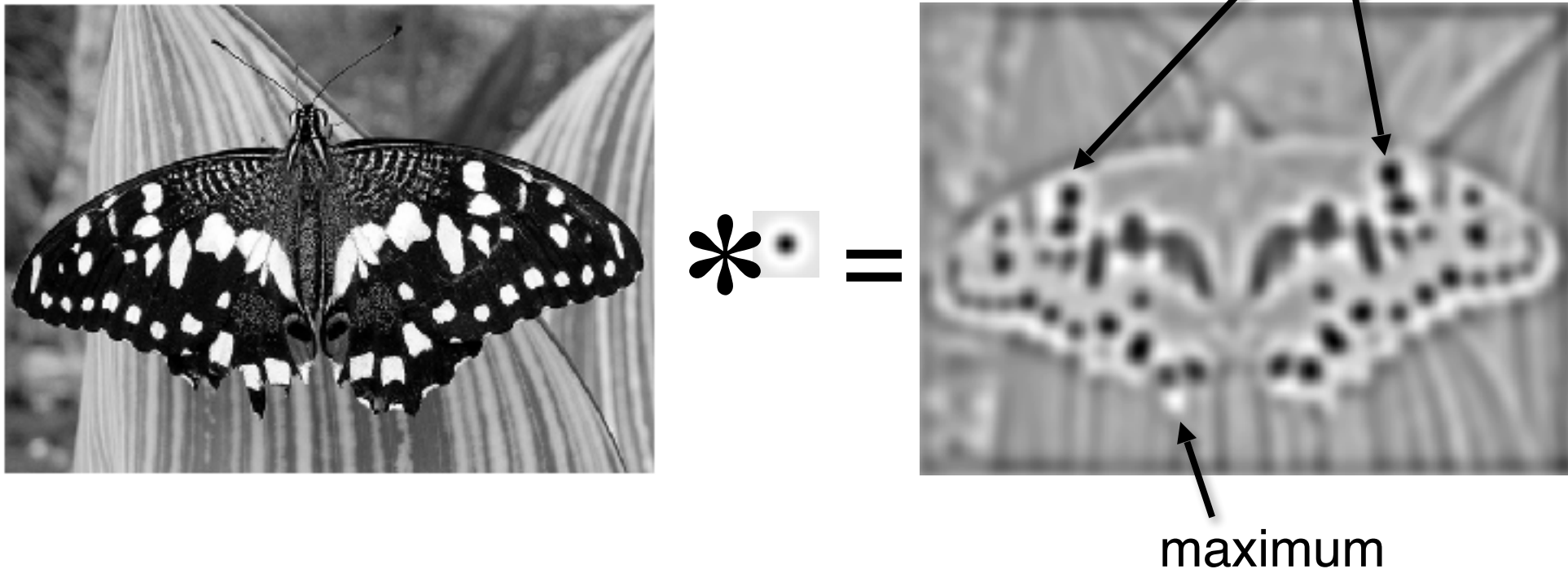
$$I_P = \begin{pmatrix} 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 120 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \\ 80 & 80 & 80 & 80 & 80 \end{pmatrix}$$

Laplace filtering

$$\begin{aligned}
 I_K'' &= \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ -40 & -40 & -40 & -40 & -40 \\ +40 & +40 & +40 & +40 & +40 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, & I_E'' &= \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ -40 & -40 & -40 & 0 & 0 \\ +40 & +40 & \textbf{+80} & -40 & 0 \\ 0 & 0 & +40 & -40 & 0 \\ 0 & 0 & +40 & -40 & 0 \end{pmatrix}, \\
 I_L'' &= \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ -40 & -40 & -40 & -40 & -40 \\ \textbf{+80} & \textbf{+80} & \textbf{+80} & \textbf{+80} & \textbf{+80} \\ -40 & -40 & -40 & -40 & -40 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, & I_P'' &= \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -40 & 0 & 0 \\ 0 & -40 & \textbf{+160} & -40 & 0 \\ 0 & 0 & -40 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.
 \end{aligned}$$

Laplacian of Gaussian

- “Blob” detector



- Find maxima *and minima* of LoG operator in space and scale

Original



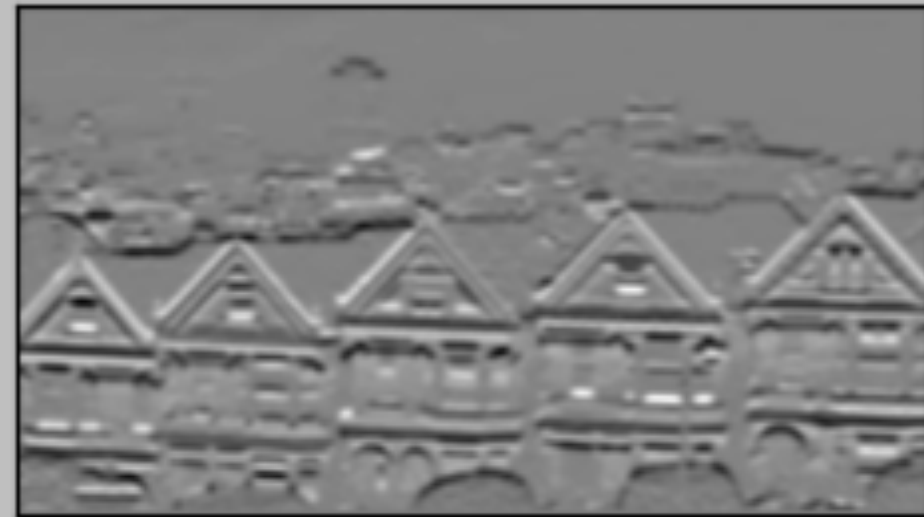
Laplacian



Sobel X

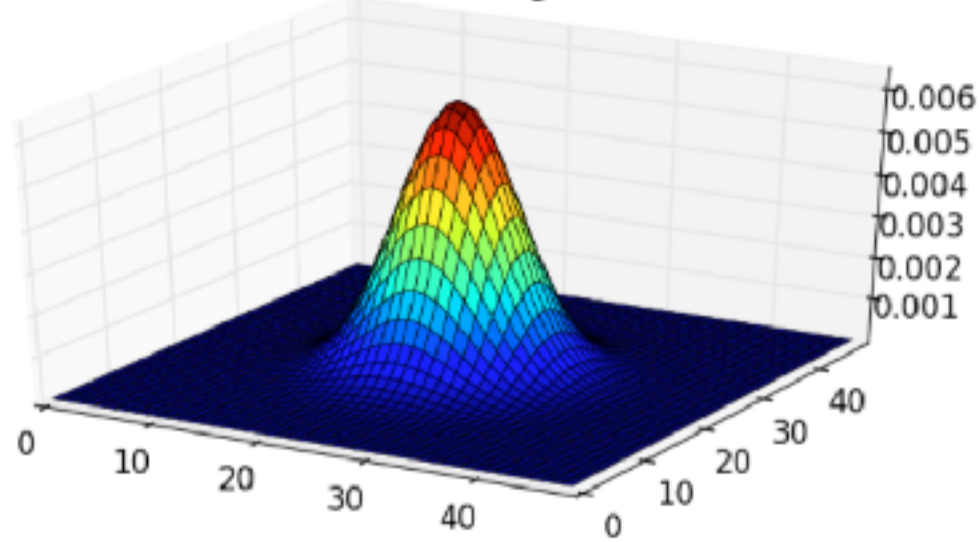


Sobel Y

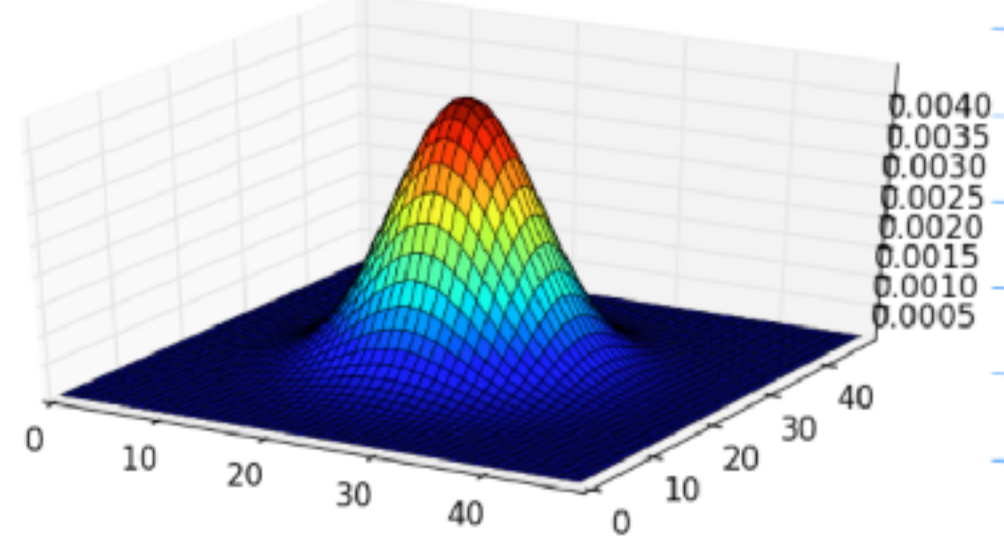


Difference of Gaussians (DoG)

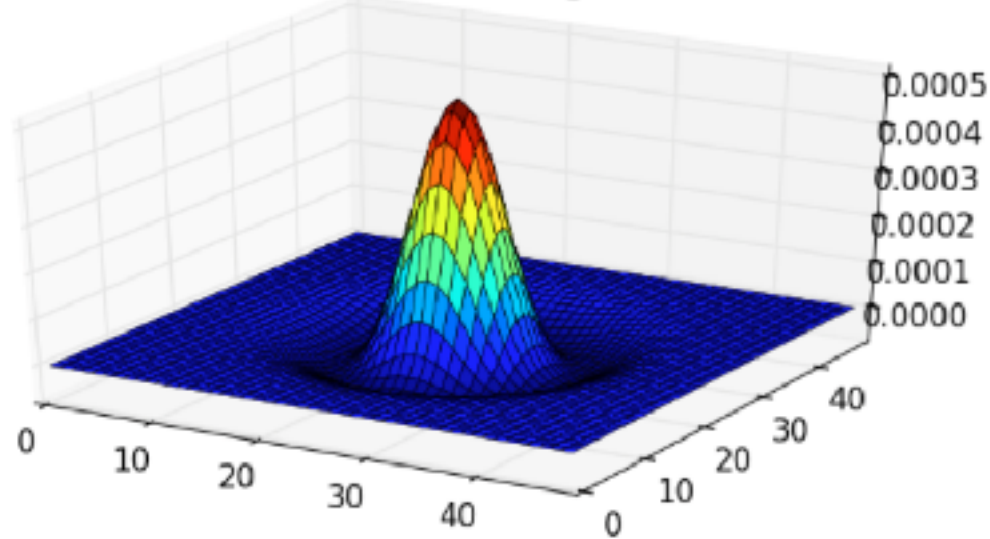
Gaussian Kernel (sigma=5.0)



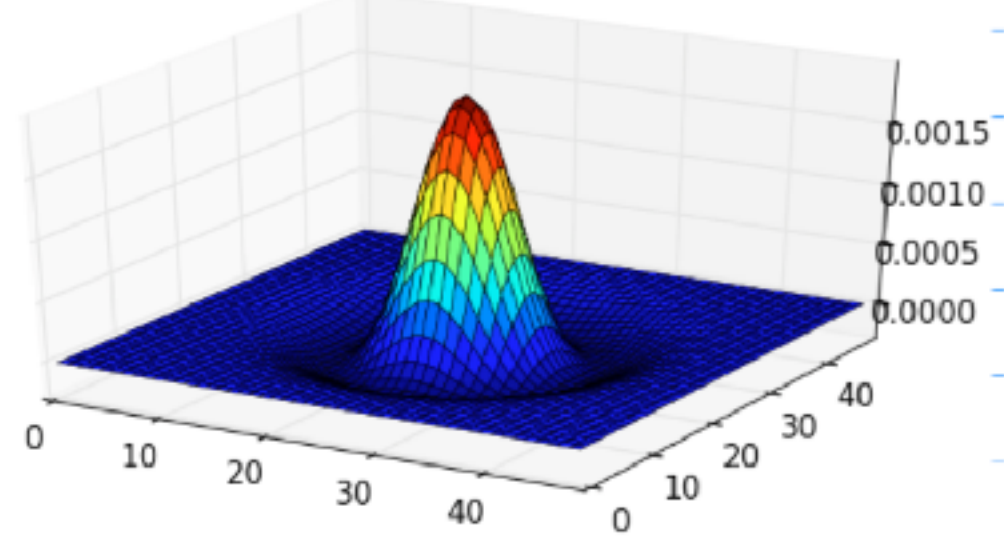
Gaussian Kernel (sigma=6.0)



Inverted LoG Kernel (sigma=5.0)



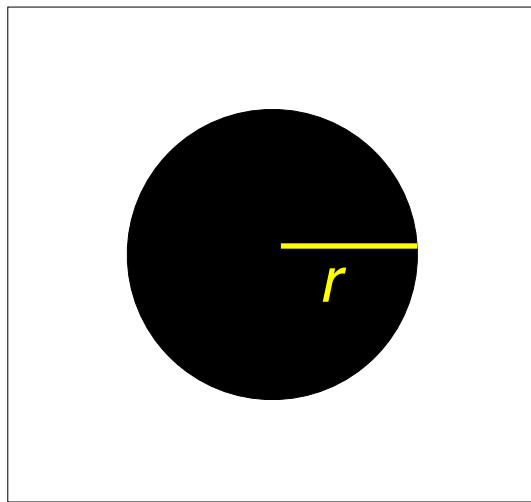
Inverted DoG Kernel (sigma1=5.0, sigma2=6.0)



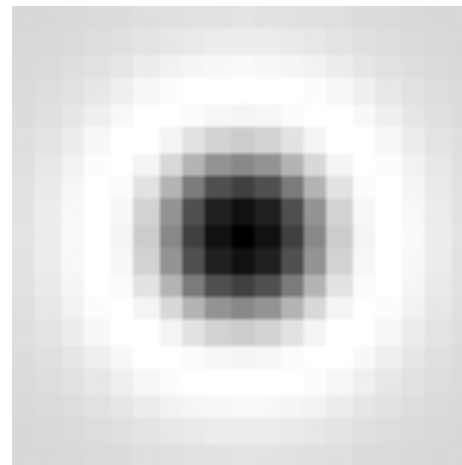
Scale selection for 2nd derivative features

Scale selection

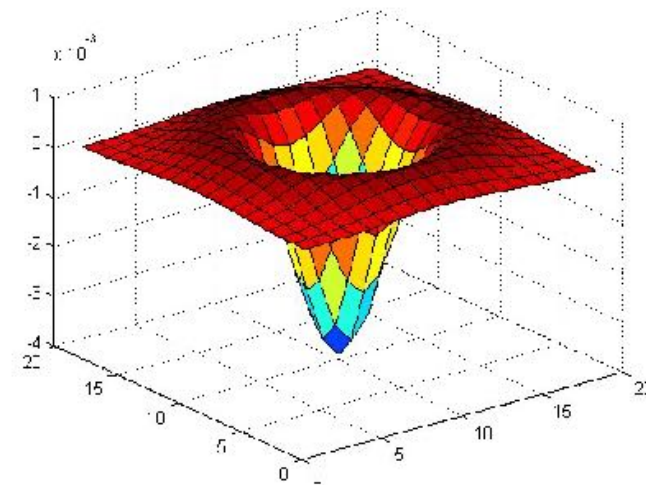
- At what scale does the Laplacian achieve a maximum response for a binary circle of radius r ?



image

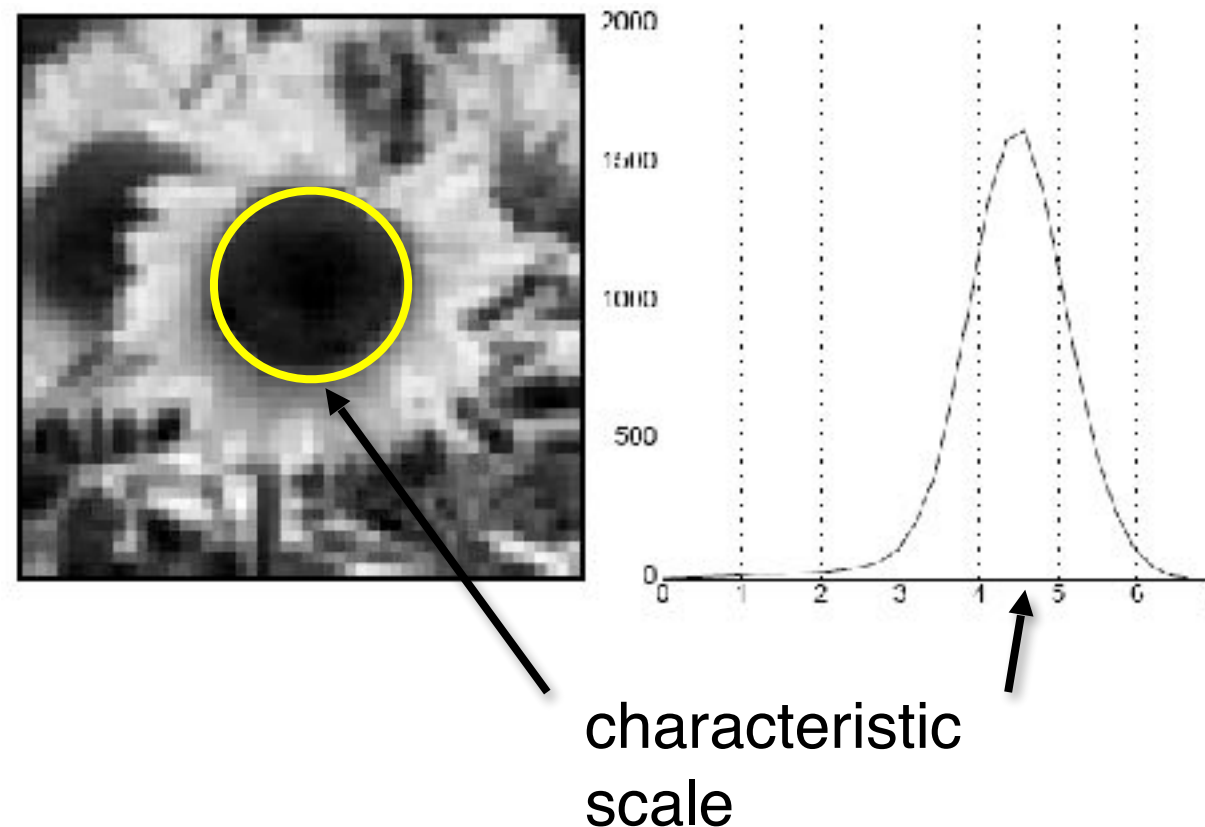


Laplacian

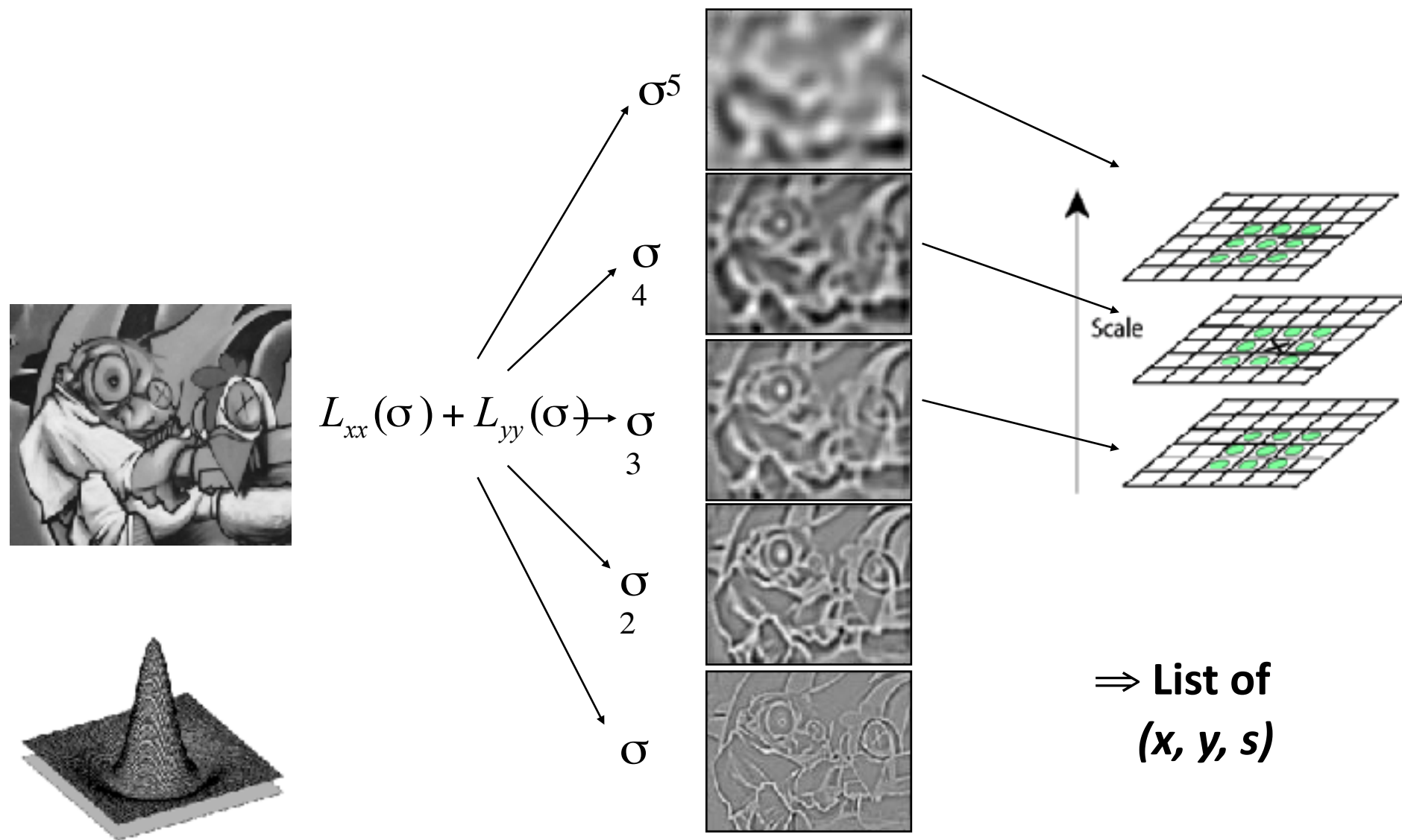


Characteristic scale

- We define the characteristic scale as the scale that produces peak of Laplacian response



Find local maxima in 3D position-scale space

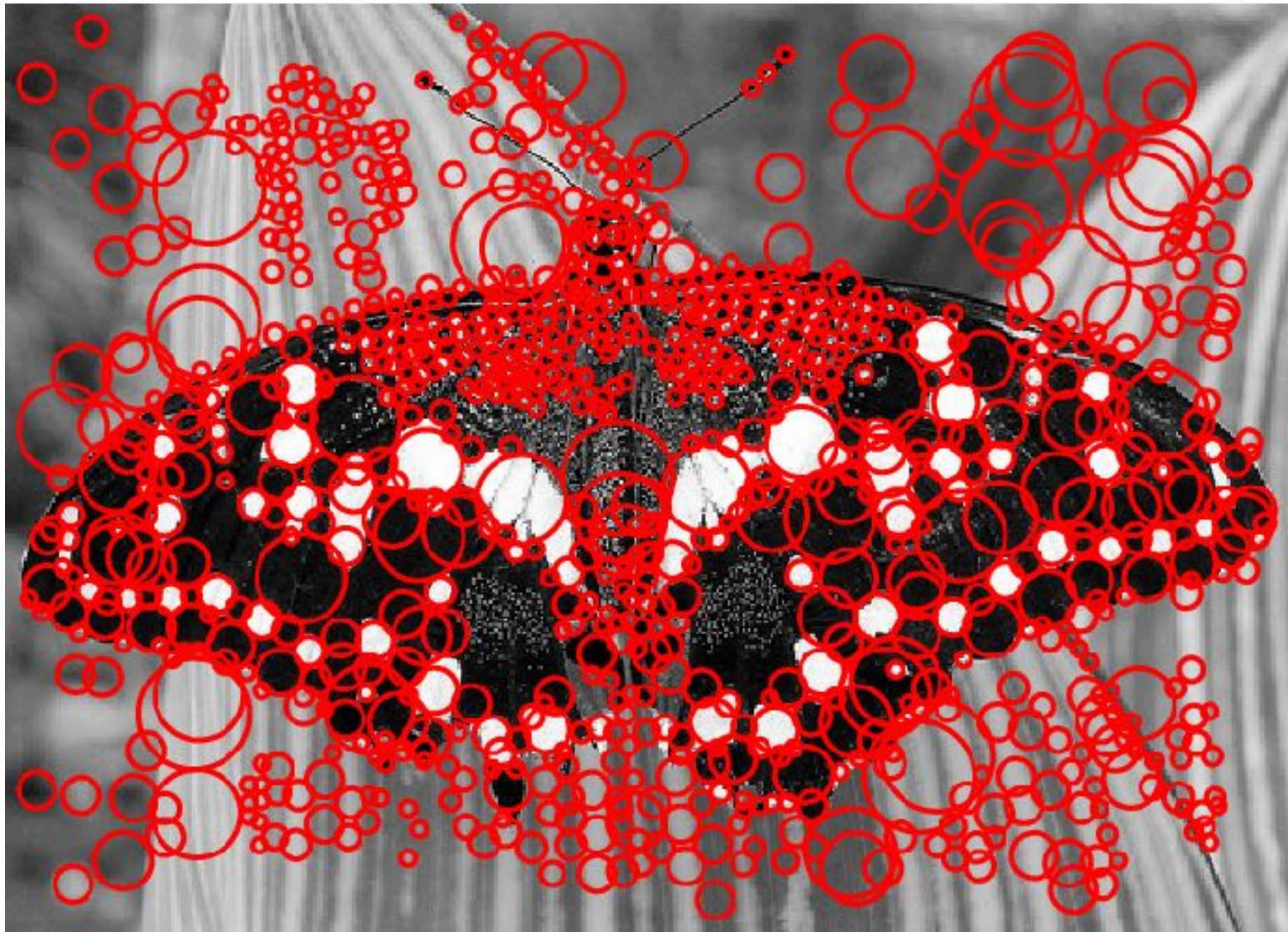


K. Grauman, B. Leibe

Scale-space blob detector: Example

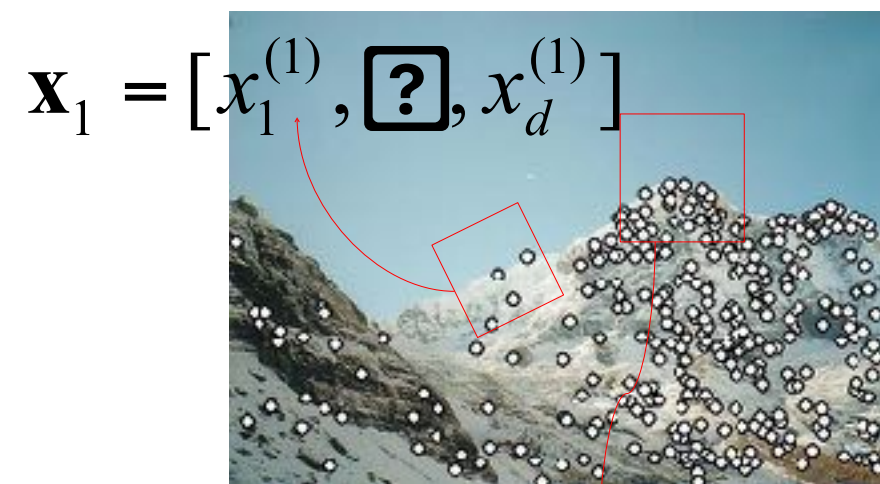


Scale-space blob detector: Example



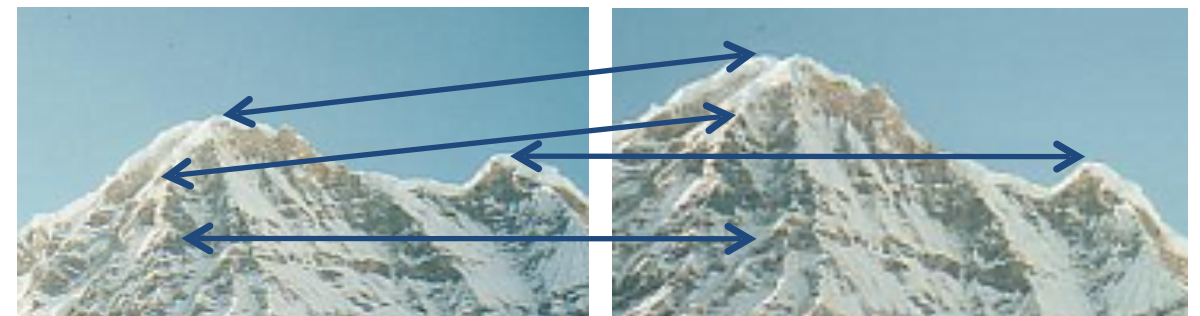
Feature detection and matching

- 1) Detection:** Identify the interest points
- 2) Description:** Extract vector feature descriptor surrounding each interest point.
- 3) Matching:** Determine correspondence between descriptors in two views



$$\mathbf{x}_1 = [x_1^{(1)}, \boxed{?}, x_d^{(1)}]$$

$$\mathbf{x}_2 = [x_1^{(2)}, \boxed{?}, x_d^{(2)}]$$



Feature descriptors: SIFT (Lowe 2004)

Object Recognition from Local Scale-Invariant Features

David G. Lowe
Computer Science Department
University of British Columbia
Vancouver, B.C., V6T 1Z4, Canada
lowe@cs.ubc.ca

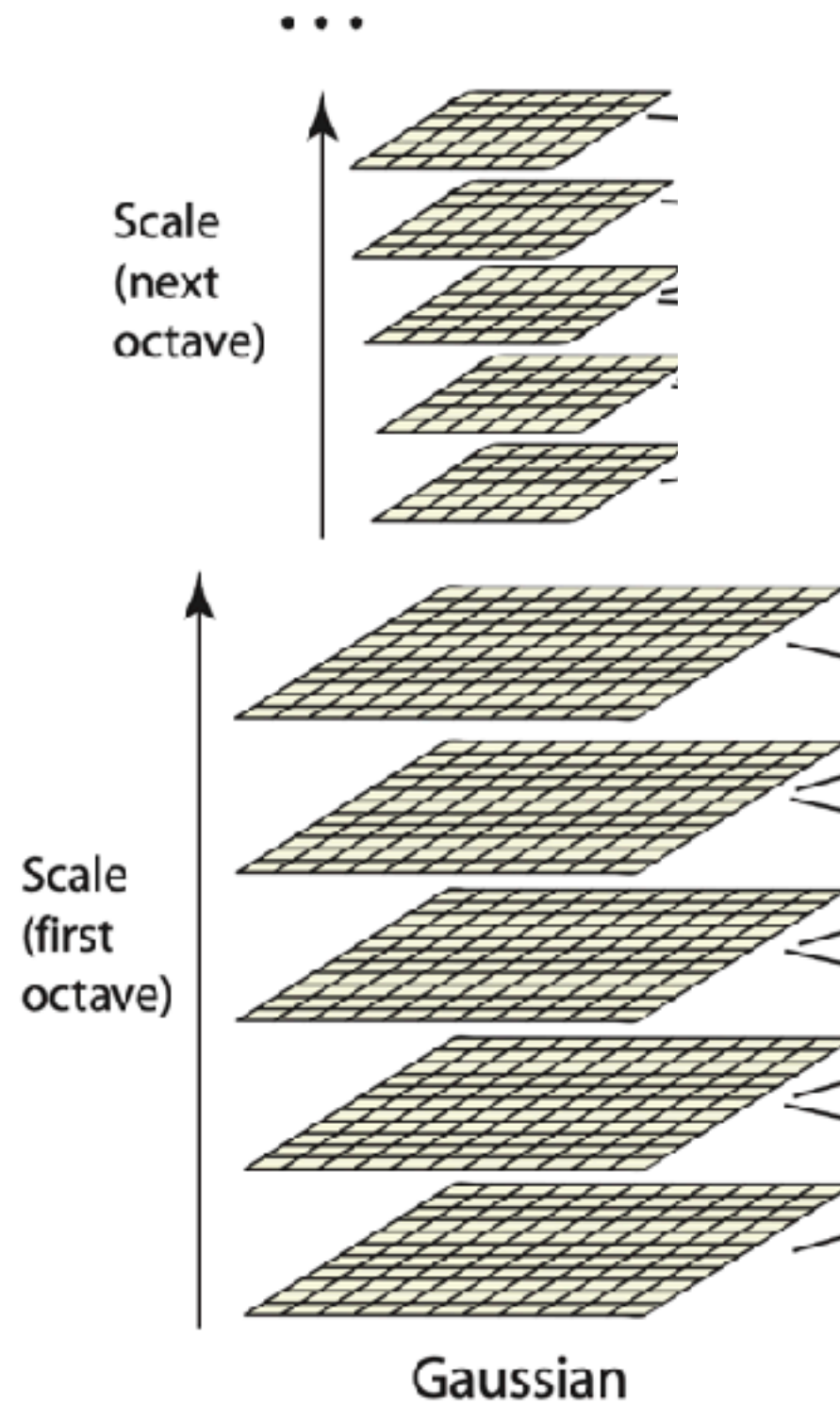
1999

Distinctive Image Features from Scale-Invariant Keypoints

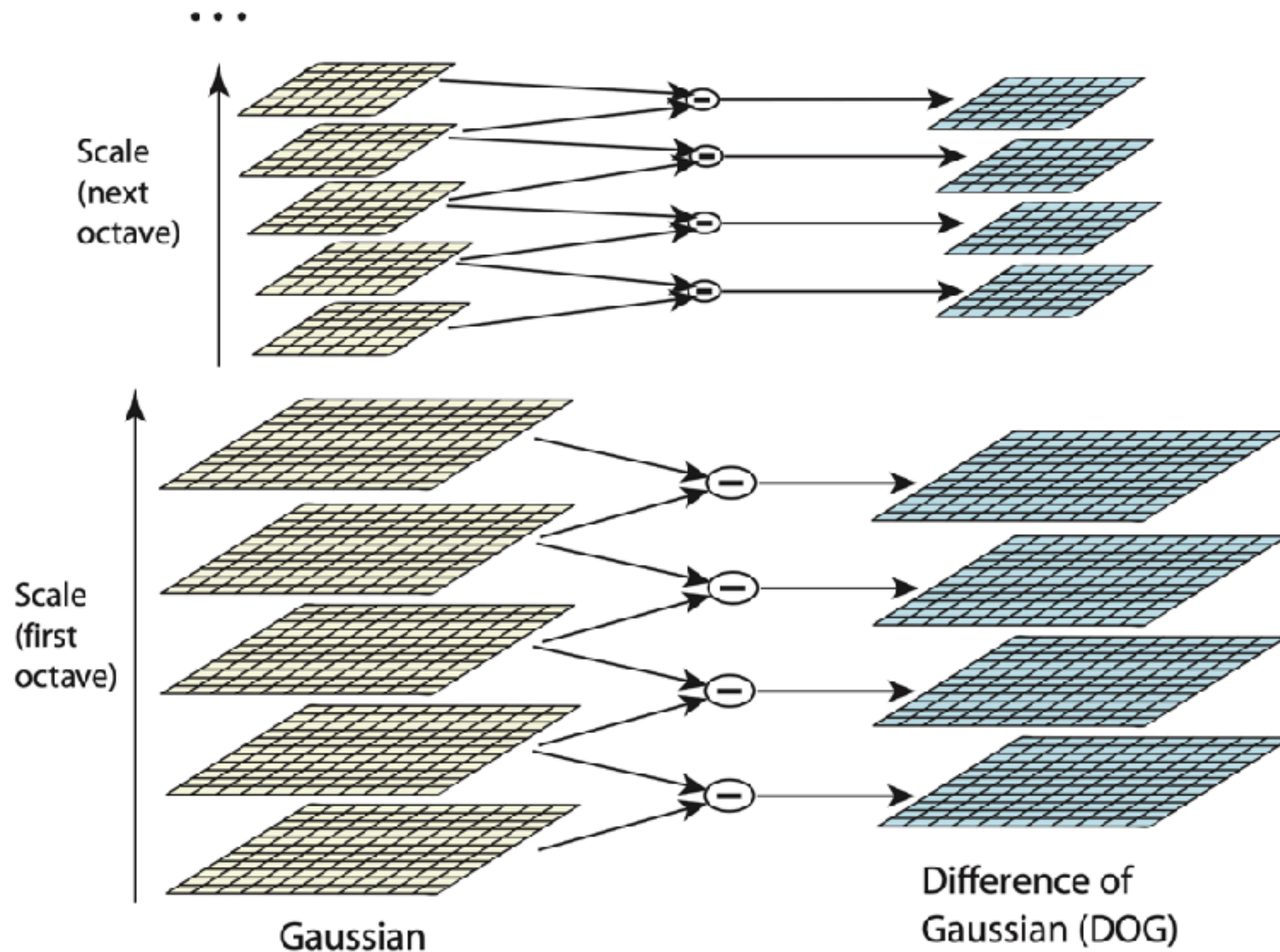
David G. Lowe
Computer Science Department
University of British Columbia
Vancouver, B.C., Canada
lowe@cs.ubc.ca

2004

1) Building a DoG scale space



2) Compute the difference of Gaussian pyramid



3) Scale invariant detection: Find local maxima in the 3D DoG pyramid

- Use points where the DoG value is (significantly) lower or higher than all neighbors in the 3x3x3 neighborhood

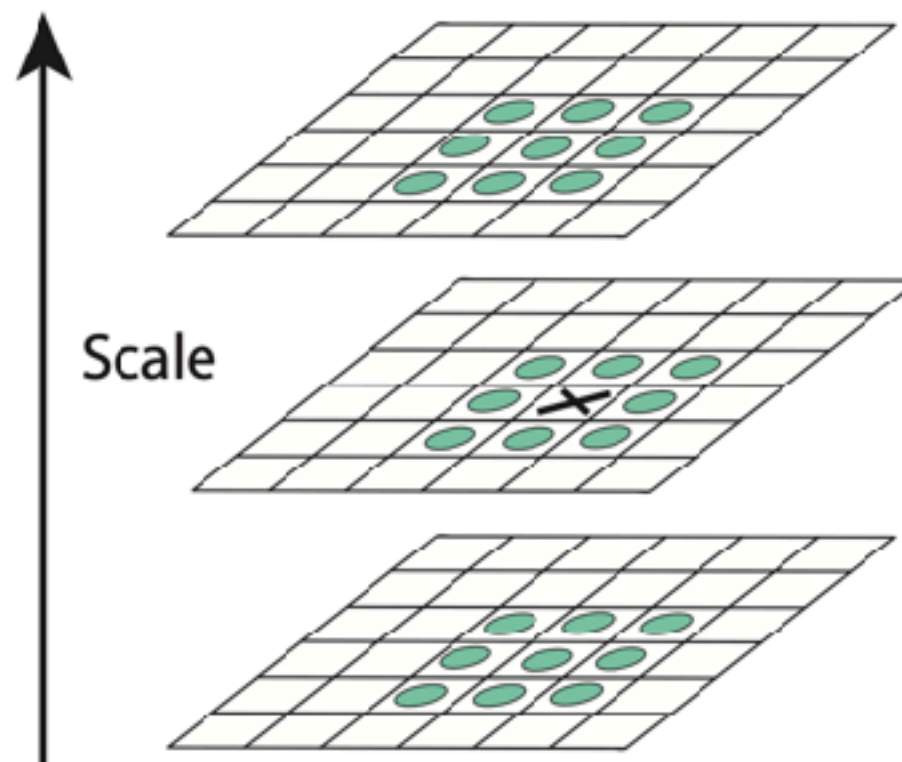
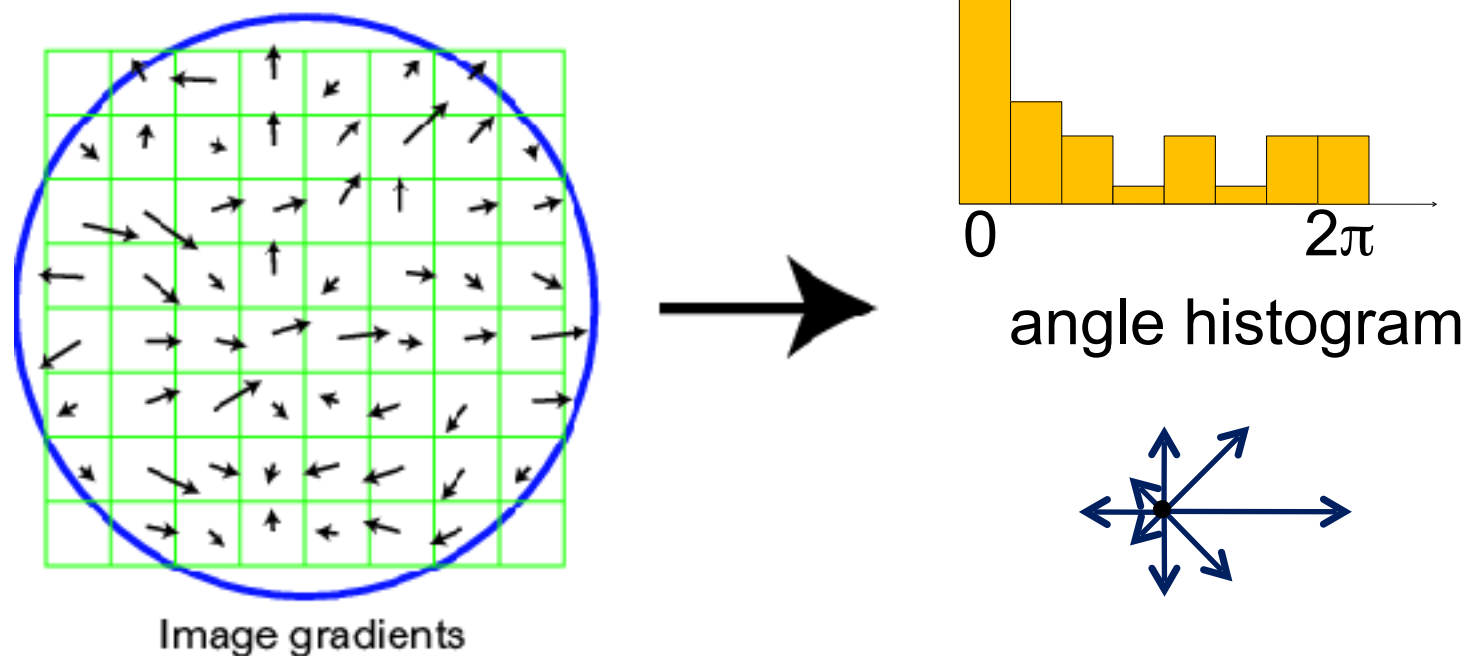


Figure 2: Maxima and minima of the difference-of-Gaussian images are detected by comparing a pixel (marked with X) to its 26 neighbors in 3x3 regions at the current and adjacent scales (marked with circles).

3) Create a descriptor for the window

Basic idea:

- Take 16x16 square window around detected feature
- Compute edge orientation (angle of the gradient - 90°) for each pixel
- Throw out weak edges (threshold gradient magnitude)
- Create histogram of surviving edge orientations



3) Create a descriptor for the window

Full version

- Divide the 16x16 window into a 4x4 grid of cells (2x2 case shown below)
- Compute an orientation histogram for each cell
- 16 cells * 8 orientations = 128 dimensional descriptor

